

NLP Project 2: Language Modelling, Sentiment Classification, and Sentence Boundary Detection

Kamal Aghazada

Fatulla Bashirov

February 2026

Abstract

This report documents Project 2 results for Task 1, Task 2, Task 3, Task 4, and the Extra Task (UI). Tasks 1, 2, and 4 use the shared poetry dataset (846 rows, 9 authors), while Task 3 uses the labeled NLTK `twitter_samples` sentiment dataset (10,000 tweets). We implemented reproducible pipelines for unsmoothed n-gram modelling, smoothing-method comparison, sentiment classification with multiple algorithms and feature sets, and sentence boundary detection with logistic regression (L1/L2 regularization). The report summarizes methodology, exact metrics, statistical testing, and implementation artifacts.

1 Team Member Contributions

Work distribution in this project was task-based:

- **Fatulla Bashirov**: implemented and analyzed **Task 1** (unsmoothed n-gram models and perplexity) and **Task 2** (smoothing method comparison and ranking).
- **Kamal Aghazada**: implemented and analyzed **Task 3** (sentiment classification with Naive Bayes, Binary Naive Bayes, Logistic Regression, and significance testing) and **Task 4** (sentence boundary detection with L1/L2 logistic regression).

Both team members reviewed final outputs, validated generated artifacts, and collaborated on report integration and UI delivery.

2 Introduction and Dataset

Project 2 uses `project2/poems_translated.parquet`, containing the columns `author`, `title`, `url`, `text`, and `modern_text`. All main experiments in this report are based on `modern_text`, with deterministic random seed 42.

The implemented deliverables are:

- Task 1: Unigram, bigram, trigram language models and perplexity.
- Task 2: Laplace, Interpolation, Backoff, and Kneser-Ney smoothing comparison.
- Task 3: Sentiment classification using Naive Bayes, Binary Naive Bayes, and Logistic Regression with Bag-of-Words and sentiment-lexicon features.
- Task 4: Dot-level sentence boundary detection with logistic regression (L1 vs L2).
- Extra Task: FastAPI web interface for interactive analysis and inference.

3 Task 1: Unsmoothed N-gram Models and Perplexity

3.1 Method

The Task 1 pipeline (`project2/task1/task1_ngram.py`) applies:

- Unicode word tokenization with regex `\b\w+\b` and lowercasing.
- Cleaning: remove empty texts and rows without tokens.
- Stratified train/test split by author (`test_size=0.2`).
- Unknown-word policy: tokens with train frequency < 2 are mapped to `<unk>`.
- Boundary handling for higher-order models with `<s>` and `</s>`.

Perplexity is computed as:

$$\text{PPL} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log p(w_i \mid h_i) \right),$$

and is reported as `inf` if any zero-probability event occurs in evaluation.

3.2 Data and Vocabulary Statistics

Table 1: Task 1 corpus and split statistics

Metric	Value
Input rows	846
Rows after cleaning/tokenization	844
Train rows	675
Test rows	169
Train tokens after <unk> mapping	112,997
Test tokens after <unk> mapping	29,942
Raw train vocabulary size	25,098
Retained vocabulary size (freq ≥ 2)	10,009
Effective vocabulary with <unk>	10,010

3.3 Perplexity Results

Table 2: Task 1 model perplexities (unsmoothed MLE)

Model	Train PPL	Test PPL	Zero-Prob Test	Unseen Rate (Test)
Unigram	1309.13	749.86	0	0.0000
Bigram	33.10	inf	14,927	0.4957
Trigram	2.33	inf	26,193	0.8699

Main observation: unsmoothed higher-order models fit training data strongly but fail on held-out text because many n-grams are unseen. This motivates smoothing in Task 2.

Top-frequency outputs (`project2/task1/results/task1_top_ngrams.csv`) also show heavy <unk> presence, confirming the long-tail lexical distribution.

4 Task 2: Smoothing Method Comparison

4.1 Method

Task 2 (`project2/task2/task2_smoothing.py`, `smoothing_core.py`) compares:

- Laplace (add-one),
- Interpolation (bigram and trigram mixtures),
- Katz-style Backoff (discount grid),
- Kneser-Ney (absolute discounting with continuation probability).

Evaluation protocol:

- Outer split: train/test with `test_size=0.2`.
- Inner split: inner-train/validation from outer-train with `val_size=0.1`.
- Tune hyperparameters on validation perplexity, retrain on final train, evaluate on test.

4.2 Split and Tuning Setup

Table 3: Task 2 split configuration

Metric	Value
Outer train rows	675
Inner train rows	606
Validation rows	69
Test rows	169
Interpolation bigram grid	0.5, 0.6, 0.7, 0.8, 0.9
Interpolation trigram step	0.1
Discount grid	0.5, 0.75, 1.0

4.3 Best Models per Method and Order

Table 4: Task 2 best tuned configurations

Method	Order	Best Params	Val PPL	Test PPL
Laplace	Bigram	$\alpha = 1.0$	2186.53	3192.02
Interpolation	Bigram	$\lambda_1 = 0.5, \lambda_2 = 0.5$	429.93	642.01
Backoff	Bigram	$d = 0.75$	410.56	600.67
Kneser-Ney	Bigram	$d = 0.75$	398.88	586.84
Laplace	Trigram	$\alpha = 1.0$	5967.74	7850.22
Interpolation	Trigram	$\lambda_1 = 0.8, \lambda_2 = 0.2, \lambda_3 \approx 0.0$	399.42	580.93
Backoff	Trigram	$d = 0.75$	421.50	634.05
Kneser-Ney	Trigram	$d = 0.75$	412.40	634.32

4.4 Ranking Results

Table 5: Best method by ranking rule

Rule	Best Method
trigram_test_ppl	Interpolation
bigram_test_ppl	Kneser-Ney
avg_bigram_trigram_test_ppl	Kneser-Ney

By the default decision rule used in the implementation (`trigram_test_ppl`), the selected best method is **Interpolation**.

5 Task 3: Sentiment Classification (Naive Bayes, Binary NB, Logistic)

5.1 Dataset and Setup

Task 3 (`project2/task3/task3_classification.py`, `classification_core.py`) uses the labeled NLTK `twitter_samples` corpus:

- Total samples: 10,000 tweets.
- Labels: `positive` and `negative`.
- Split: stratified 80/20 train/test with seed 42.
- Train/Test sizes: 8,000 / 2,000.

Evaluated classifiers:

- Multinomial Naive Bayes,
- Binary Naive Bayes,
- Logistic Regression.

Feature sets:

- **BoW**: CountVectorizer word features,
- **Lexicon**: opinion lexicon counts/ratios + VADER polarity features,
- **BoW+Lexicon**: concatenation of both.

5.2 Main Results

Table 6: Task 3 ranking by macro F1 (top configurations)

Rank	Classifier + Features	Accuracy	Macro Precision	Macro F1
1	Logistic Regression + BoW+Lexicon	0.7990	0.7992	0.7990
2	Logistic Regression + BoW	0.7930	0.7938	0.7929
3	Naive Bayes + BoW+Lexicon	0.7870	0.7873	0.7869
4	Binary Naive Bayes + BoW+Lexicon	0.7850	0.7851	0.7850

Classifier-level best configurations:

- Logistic Regression best: **BoW+Lexicon**, macro F1 = 0.79897.
- Naive Bayes best: **BoW+Lexicon**, macro F1 = 0.78695.
- Binary Naive Bayes best: **BoW+Lexicon**, macro F1 = 0.78499.

5.3 Statistical Significance Testing

We used pairwise McNemar tests on the same test predictions ($\alpha = 0.05$). Key comparisons:

- Logistic (BoW+Lexicon) vs Naive Bayes (BoW): $\chi^2 = 5.0039$, $p = 0.02529$, significant.
- Logistic (BoW+Lexicon) vs Naive Bayes (BoW+Lexicon): $\chi^2 = 1.9593$, $p = 0.16159$, not significant.
- Logistic (BoW+Lexicon) vs Binary NB (BoW+Lexicon): $\chi^2 = 2.4966$, $p = 0.11409$, not significant.

5.4 Analysis

The best overall model on this split is **Logistic Regression with BoW+Lexicon**. However, the significance tests show that its advantage over the strongest Naive Bayes variants is modest and not statistically significant in all pairwise comparisons. Therefore, the most accurate conclusion is:

- Logistic Regression is the top-performing classifier in point estimates,
- but Logistic and the best Naive Bayes variants are competitive under paired significance testing.

6 Task 4: Sentence Boundary Detection (L1 vs L2)

6.1 Method

Task 4 (`project2/task4/task4_sentence_boundary.py`, `sentence_boundary_core.py`) models each dot character (.) as a binary decision: sentence boundary vs not boundary.

Pipeline highlights:

- Dot-level sample extraction from each poem.
- Heuristic labeling excludes ellipses and decimal numbers.
- 18 engineered features (contextual, positional, lexical, abbreviation and punctuation cues).
- Split by poem ID (not by dot) into train/val/test to avoid leakage across dots from the same poem.
- Logistic regression with **saga** solver, comparing L1 and L2 penalties.
- Grid search over $C \in \{0.001, 0.01, 0.1, 0.5, 1.0, 5.0, 10.0\}$ on validation F1.

6.2 Dataset Characteristics

Table 7: Task 4 dot-level dataset statistics

Metric	Value
Total non-empty texts	844
Total dot samples	9,428
Positive dots (boundary)	9,191
Negative dots (not boundary)	237
Positive rate	0.9749
Train / Val / Test dots	6,490 / 875 / 2,063

6.3 L1 vs L2 Results

Table 8: Task 4 final comparison (best C per penalty)

Penalty	Best C	Train F1	Val F1	Test F1	Test Acc.	Test Recall
L1	10.0	0.998355	0.997585	0.983094	0.968008	0.969192
L2	0.01	0.997810	0.997590	0.997477	0.995153	0.998485

Test-time McNemar comparison between L1 and L2:

$$\chi^2 = 48.7903, \quad p = 2.8484 \times 10^{-12},$$

which is statistically significant at $\alpha = 0.05$. The selected best penalty is **L2**.

6.4 Feature Analysis

Top absolute coefficients show stable cues across both penalties:

- `is_ellipsis` (strong negative),
- `next_is_quote` (negative),
- `prev_char_is_alpha` (positive),
- `next_char_is_space` (negative),
- `word_len` (positive).

This indicates that punctuation context and immediate local token structure are the strongest predictors for dot-boundary decisions.

7 Extra Task: FastAPI UI Deliverables

A unified FastAPI interface was implemented in `fastapi_ui/` with a single-page HTML/CSS/JS frontend and API endpoints for all tasks.

Main UI/backend files:

- `fastapi_ui/main.py`
- `fastapi_ui/templates/index.html`
- `fastapi_ui/static/app.js`
- `fastapi_ui/static/styles.css`

The UI provides:

- left-side task navigation and per-task Input/Output/Implementation panels,
- dataset/split parameter controls,
- metric dashboards and comparison tables,
- custom text inference (next-word prediction for Tasks 1–2, sentiment prediction for Task 3, sentence detection for Task 4),
- confusion matrices and significance tables for classification tasks.

To improve responsiveness for heavy tasks, the FastAPI app includes in-memory and persistent disk caching (`fastapi_ui/.model_cache`) and a warmup endpoint (`/api/warmup`) to prebuild Task 2–4 artifacts.

8 Discussion and Conclusion

The Project 2 results show a consistent pattern:

1. Unsmoothed higher-order n-gram models are not robust on held-out poetic text, despite strong train fit.
2. Smoothing is essential. Interpolation gives the best trigram test perplexity, while Kneser-Ney is strongest for bigram and average rules.
3. For sentiment classification on `twitter_samples`, Logistic Regression with BoW+Lexicon gives the best overall macro F1, while top Naive Bayes variants remain competitive in paired significance tests.
4. Dot-level sentence boundary detection works well with carefully designed local features; L2 regularization significantly outperforms L1 on test data in this setup.
5. The FastAPI interface makes each task reproducible, interactive, and easier to inspect qualitatively.

Future improvements should include stronger handling of class imbalance in Task 4 and richer domain-specific lexical resources for language modelling and downstream punctuation decisions.